Introdução à Orientação a Objetos

Aula 10 — Composição e herança (parte 1)

Programação Orientada a Objetos

Semana 10

Sumário

1. Revisão: composição
2. O problema: livros digitais
3. Herança com `extends`
4. Construtores e `super`
5. O que a subclasse enxerga
6. `LivroFisico`
7. Regras da herança em Java
8. Herança ou composição?
9. Exercício de fixação
10. Fechamento

Objetivos da aula

Ao final desta aula você deve ser capaz de:

- distinguir as relações **"tem um"** (composição) e **"é um"** (herança)
- criar uma subclasse com `extends`
- usar `super(...)` para chamar o construtor da superclasse
- explicar o que a subclasse herda e o que ela **não** enxerga
- prever a ordem de execução dos construtores numa hierarquia

Vocês já usam composição

Na aula 09, a classe `Livro` tinha estes atributos:

```
1 public class Livro {
2     private String nome;
3     private String isbn;
4     private double valor;
5     private LocalDate dataPublicacao;
6     private Autor autor;
```

- Um atributo do tipo `Autor`: o `Livro` **tem um** `Autor`
- "Uma classe pode ter outra classe como atributo, esse é um processo natural conhecido como **composição**" (TURINI, [20–], seção 3.3)

Composição também com listas

```java
public class Catalogo {
    private List<Livro> livros = new ArrayList<>();
```

```java
public class CarrinhoDeCompras {
    private List<Livro> livros = new ArrayList<>();
```

- O `Catalogo` **tem vários** `Livro`
- O carrinho **tem vários** `Livro`
- Composição = montar um objeto a partir de outros objetos

A livraria vai vender e-books

- Muitos leitores preferem livros digitais (*e-books*)
- Um e-book tem tudo o que um livro tem: nome, ISBN, valor, autor...
- Mas tem coisas só dele: a **marca d'água** (*watermark*), com nome e e-mail do dono
- E regras diferentes: desconto máximo de 15% (o livro impresso aceita 30%)

Como representar isso no código?

Fonte: Turini ([20–], seção 5.1).

Ideia 1: um boolean e mais um if

Contraexemplo (não faça assim):

```
1  private boolean impresso;
2
3  public boolean aplicaDescontoDe(double porcentagem) {
4      if (porcentagem > 0.3) {
5          return false;
6      } else if (!this.impresso && porcentagem > 0.15) {
7          return false;
8      }
9      this.valor -= this.valor * porcentagem;
10     return true;
11 }
```

- Cada regra diferente pede um novo `if`
- Cada novo tipo de livro (audiolivro?) piora tudo
- E onde fica o `waterMark`? Num livro impresso ele não faz sentido

Ideia 2: copiar a classe

Contraexemplo (não faça assim):

```
1 public class Ebook {
2     private String nome;
3     private String isbn;
4     private double valor;
5     private Autor autor;
6     private String waterMark;
7     // ... e TODOS os metodos de Livro copiados aqui
```

- Agora `Ebook` é um objeto próprio: bom
- Mas é **código repetido**: corrigiu um *bug* em `Livro`, tem de lembrar de corrigir em `Ebook`

Queremos dizer ao compilador: **um** `Ebook` **é um** `Livro`, com algo a mais.

extends: um Ebook é um Livro

```java
public class Ebook extends Livro {
    private String waterMark;

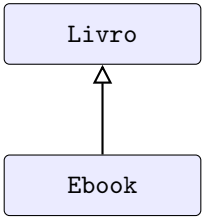
    public Ebook(Autor autor) {
        super(autor);
    }

    public String getWaterMark() {
        return waterMark;
    }

    public void setWaterMark(String waterMark) {
        this.waterMark = waterMark;
    }
}
```

Arquivo: `produtos/Ebook.java`. Adaptado de Turini ([20–], seção 5.1).

Vocabulário



A seta com triângulo vazado aponta para a classe mãe.

- `Livro`: **superclasse** (classe mãe, classe pai)
- `Ebook`: **subclasse** (classe filha)
- O `Ebook` **herda** os atributos e métodos de `Livro`
- E acrescenta os seus: `waterMark`, `getWaterMark`, `setWaterMark`

Fontes: Turini ([20–], seção 5.1); Schildt (2022, cap. 7).

O Ebook já sabe fazer tudo o que o Livro faz

```
1 Ebook ebook = new Ebook(aniche);
2 ebook.setNome("Test-Driven Development");
3 ebook.setValor(29.90);
4 ebook.setIsbn("978-85-5519-024-8");
5 ebook.setWaterMark("aluno@ifsp.edu.br");
6 ebook.mostrarDetalhes();
7 System.out.println("Marca d'agua: " + ebook.getWaterMark());
```

- `setNome`, `setValor`, `setIsbn`, `mostrarDetalhes`: escritos **uma vez**, em `Livro`
- `setWaterMark`, `getWaterMark`: só existem em `Ebook`

Saída

```
Nome: TEST-DRIVEN DEVELOPMENT
ISBN: 9788555190248
Valor: R$ 29,90
Autor: Mauricio Aniche
--
Marca d'agua: aluno@ifsp.edu.br
```

- O nome saiu em maiúsculas: o `setNome` herdado é o da aula 09
- O ISBN saiu sem hífens: mesma coisa com o `setIsbn`

Por que o `super(autor)`?

O construtor de `Livro` exige um `Autor`:

```
1 public Livro(Autor autor) {
2     this.autor = autor;
3     totalCadastrados++;
4 }
```

- Ao herdar de `Livro`, o `Ebook` herdou também essa obrigação
- `super(autor)` **delega** a construção da parte "livro" para a superclasse
- Deve ser a **primeira linha** do construtor da subclasse

Fontes: Turini ([20–], seção 5.1, "delegate constructor"); Schildt (2022, cap. 7, "Using super to Call Superclass Constructors").

E se esquecermos o super?

```java
public class Ebook extends Livro {
    private String waterMark;

    public Ebook(String waterMark) {
        this.waterMark = waterMark;
    }
}
```

```
error: constructor Livro in class Livro cannot be applied to given types;
  required: Autor
  found: no arguments
```

- Sem `super(...)`, o Java tenta chamar `super()` **sem argumentos**
- `Livro` não tem construtor sem argumentos: erro de compilação

O contador de Livro conta as filhas

```
1 Ebook ebook = new Ebook(aniche);
2 LivroFisico fisico = new LivroFisico(aniche);
3 System.out.println("Livros cadastrados: " + Livro.getTotalCadastrados());
```

```
Livros cadastrados: 2
```

- Criar um Ebook executa o construtor de Livro (via super)
- Por isso o totalCadastrados++ rodou duas vezes
- **Pergunta:** e se o construtor de Ebook não chamasse super? Compilaria?

private continua private

```java
public class Ebook extends Livro {
    public void mostraValor() {
        System.out.println(this.valor);
    }
}
```

```
error: valor has private access in Livro
```

- O Ebook **tem** um valor (herdou o atributo), mas **não enxerga** o atributo diretamente
- "Um atributo private só pode ser acessado pela própria classe, nem mesmo as classes filhas podem violar essa regra" (TURINI, [20–], seção 5.2)

Solução: use os métodos da superclasse

```java
public void mostraValor() {
    System.out.println(this.getValor());
}
```

- O encapsulamento de `Livro` fica intacto
- Existe o modificador `protected` (visível para as filhas), mas ele afrouxa o encapsulamento: **vamos evitá-lo**

Fontes: Turini ([20–], seção 5.2); Schildt (2022, cap. 7, "Member Access and Inheritance").

Regras só do livro físico

Todo livro físico tem uma taxa de impressão de 5%. Onde colocar esse método?

- Em `Livro`? Então o `Ebook` herdaria uma taxa que não existe para ele
- Solução: outra subclasse

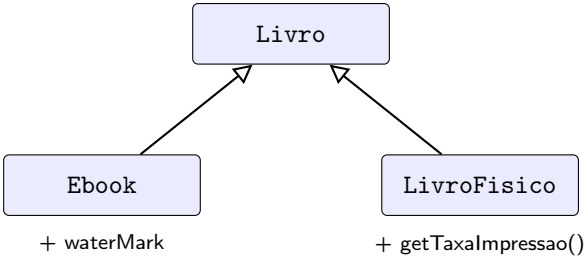
```java
public class LivroFisico extends Livro {
    public static final double TAXA_IMPRESSAO = 0.05;

    public LivroFisico(Autor autor) {
        super(autor);
    }

    public double getTaxaImpressao() {
        return this.getValor() * TAXA_IMPRESSAO;
    }
}
```

Adaptado de Turini ([20–], seção 5.3).

A hierarquia até agora



- "Então eu devo criar uma nova classe pra cada tipo de livro?" **Sim**
- Classes pequenas, com regras próprias bem encapsuladas, são mais fáceis de manter do que uma classe gigante cheia de `if`

Fonte: Turini ([20–], seção 5.3).

Testando o LivroFisico

```
1 LivroFisico fisico = new LivroFisico(aniche);
2 fisico.setNome("Test-Driven Development");
3 fisico.setValor(59.90);
4 fisico.setIsbn("978-85-5519-024-8");
5 System.out.println("Taxa de impressao: " + String.format("R$ %.2f", fisico.
      getTaxaImpressao()));
```

```
Taxa de impressao: R$ 3,00
```

- **Pergunta:** `ebook.getTaxaImpressao()` compila? Por quê?

Só uma classe mãe

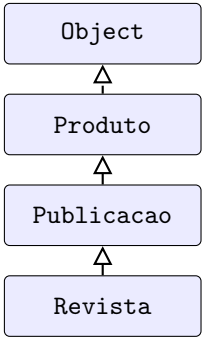
```
1 public class Multi extends Livro, Editora {
2 }
```

```
error: '{' expected
public class Multi extends Livro, Editora {
                                  ^
```

- Em Java, uma classe herda diretamente de **uma única** classe
- Não existe herança múltipla de classes (como existe em C++)

Fontes: Turini ([20–], seção 5.1, "Herança múltipla"); Schildt (2022, cap. 7).

Mas pode haver vários níveis



- `Revista` é uma `Publicacao`, que é um `Produto`
- Hierarquia em vários níveis é permitida (SCHILDT, 2022, cap. 7, "Creating a Multilevel Hierarchy")
- Toda classe sem `extends` herda implicitamente de `Object`
- Cuidado: cadeias longas aumentam o **acoplamento** (TURINI, [20–], seção 5.1)

Em que ordem os construtores rodam?

```java
class Produto {
    Produto() {
        System.out.println("1. construtor de Produto");
    }
}

class Publicacao extends Produto {
    Publicacao() {
        System.out.println("2. construtor de Publicacao");
    }
}

class Revista extends Publicacao {
    Revista() {
        System.out.println("3. construtor de Revista");
    }
}
```

Antes de rodar: o que `new Revista()` imprime?

Da mãe para a filha

```
1. construtor de Produto
2. construtor de Publicacao
3. construtor de Revista
```

- Os construtores executam na ordem da derivação: da superclasse para a subclasse
- Motivo: a subclasse pode depender da parte da mãe já pronta
- Aqui ninguém escreveu `super()`: o compilador insere `super()` sem argumentos

Fonte: Schildt (2022, cap. 7, "When Are Constructors Executed?"). Arquivo: `exemplos/OrdemDosConstrutores.java`.

O teste do "é um" e do "tem um"

Frase	Faz sentido?	Relação
Um `Ebook` **é um** `Livro`	sim	herança
Um `Livro` **tem um** `Autor`	sim	composição
Um `Livro` **é um** `Autor`	não	—
Um `Catalogo` **tem vários** `Livro`	sim	composição
Um `Catalogo` **é um** `CarrinhoDeCompras`	não	—

- Se a frase com "é um" soa estranha, **não use herança**

Mais composição: Livro tem uma Editora

```java
public class Editora {
    private String nome;
    private String cidade;

    public Editora(String nome, String cidade) {
        this.nome = nome;
        this.cidade = cidade;
    }
    // getters
}
```

```java
// em Livro
private Editora editora;
```

Exemplo próprio da disciplina, no estilo do livro-base.

A filha herda a composição

```
1 Editora casaDoCodigo = new Editora("Casa do Codigo", "Sao Paulo");
2 fisico.setEditora(casaDoCodigo);
3 fisico.mostrarDetalhes();
```

```
Nome: TEST-DRIVEN DEVELOPMENT
ISBN: 9788555190248
Valor: R$ 59,90
Autor: Mauricio Aniche
Editora: Casa do Codigo (Sao Paulo)
--
```

- O atributo `editora` foi escrito em `Livro`: `LivroFisico` e `Ebook` ganham de graça

Prefira composição

Recomendação do livro-base

"Sempre que possível procure favorecer o uso da composição entre classes no lugar de utilizar herança" (TURINI, [20–], seção 5.6).

- Herança aumenta o **acoplamento**: mudar a mãe mexe em todas as filhas
- Herança expõe a filha aos detalhes da mãe e acaba comprometendo o **encapsulamento**
- Use herança quando existir de fato um "é um" e comportamento a reaproveitar

Lista da aula 10 (em duplas, 60 min)

1. "É um", "tem um" ou nenhum?
2. Ordem dos construtores
3. Encontre e corrija os erros
4. `Ebook` e `LivroFisico` no seu projeto (checkpoint)
5. Composição: `Editora`
6. Bônus: `Audiolivro`

- Projeto de partida: a solução da aula 09
- Quem digita troca a cada questão

Um problema para a próxima aula

```
1 Ebook ebook = new Ebook(aniche);
2 ebook.setValor(29.90);
3 System.out.println(ebook.aplicaDescontoDe(0.3));
4 System.out.println(ebook.getValor());
```

```
true
20.93
```

- O e-book aceitou 30% de desconto, mas a regra dele é **no máximo 15%**
- Ele herdou o método `aplicaDescontoDe` de `Livro`, com a regra errada
- Como dar ao `Ebook` a sua própria versão do método? Na aula 11

Exit ticket

Em uma folha, com o nome da dupla:

1. Dê um exemplo de "é um" e um de "tem um" fora da livraria.
2. Por que a classe filha precisa chamar `super(autor)`?
3. O `Ebook` tem um atributo `valor`? Ele consegue escrever `this.valor`?

Resumo

- **Composição** (“tem um”): um objeto tem outro como atributo
- **Herança** (“é um”): `class Filha extends Mae`
- A filha herda atributos e métodos e acrescenta os seus
- `super(...)`: primeira linha do construtor, chama o construtor da mãe
- Atributos `private` da mãe: use getters e setters
- Uma única mãe direta; vários níveis permitidos; tudo herda de `Object`
- Construtores rodam da mãe para a filha
- Na dúvida, prefira composição

Próxima aula

- Reescrita de métodos: `@Override`
- `super.metodo()`: reaproveitar a regra da mãe
- Um carrinho que aceita qualquer tipo de `Livro`: introdução ao polimorfismo

Referências

SCHILDT, Herbert. **Java**: a beginner's guide. 9. ed. Nova York: McGraw Hill, 2022. (cap. 7)

TURINI, Rodrigo. **Desbravando Java e Orientação a Objetos**: um guia para o iniciante da linguagem. [S.l.: s.n.], [20–]. (seções 3.3, 5.1, 5.2, 5.3 e 5.6)